САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №10
«Разработка сценариев нагрузочного тестирования»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Написать сценарии и профили нагрузки (k6, Locust или Artillery) для ключевых эндпоинтов микросервисов.
2. Определить базовые метрики производительности (RPS, latency p95/p99, error rate) и целевые SLA/SLO системы.

Ход работы

1. Ключевые эндпоинты

Ключевыми выбраны эндпоинты основного пользовательского пути. Учитывалась и цена запроса: какие сервисы он затрагивает и сколько стоит по CPU.

Эндпоинт	Цепочка сервисов	Почему ключевой
`GET /restaurants` (фильтры, пагинация)	gateway → catalog → review (рейтинги batch)	главная страница поиска, самый частый запрос; fan-out в другой сервис
`GET /restaurants/{id}`, `/menu`, `/reviews`	gateway → catalog / review → auth (имена)	карточка ресторана
`GET /restaurants/{id}/tables?date&time`	gateway → catalog → reservation (занятость batch)	шаг перед бронью, два сервиса и две БД
`POST /auth/login`	gateway → auth (bcrypt, cost 10)	самый дорогой по CPU запрос системы
`POST /reservations` + `DELETE /reservations/{id}`	gateway → reservation → catalog; событие в RabbitMQ	бизнес-операция: запись в БД, проверка занятости, асинхронное событие

2. Инструмент и модель нагрузки

Инструмент — **k6** (v1.5): сценарии на JavaScript, несколько параллельных сценариев с разной интенсивностью, пороги (thresholds) прямо в коде и remote write метрик в Prometheus.

Используется **открытая модель нагрузки** (executor'ы `constant-arrival-rate` / `ramping-arrival-rate`). k6 подаёт заданное число итераций в секунду независимо от скорости ответов. В закрытой модели (фиксированное число VU, «запрос — ответ — следующий запрос») медленная система сама снижает поток запросов, и

деградация маскируется. В открытой модели она сразу видна по росту задержек и `dropped_iterations`.

Где работает генератор. k6 запускается с рабочего ноутбука и ходит к стенду через интернет: VPN-туннель → публичный порт 8088 → edge-nginx → Ingress. Так нагрузка не отнимает CPU у тестируемого сервера, который к тому же общий. Перед тестами путь откалиброван запросами к статической странице nginx (`/nginx-health`, приложение не участвует):

```
calib: ramping 20 → 200 запросов/с, 80 с
http_req_duration: avg=101.81ms med=95.9ms p(95)=122.87ms p(99)=228.44ms
http_req_failed:   0.00%  0 out of 9200
```

Путь выдерживает 200 RPS без ошибок, медиана задержки ~96 мс — это сетевой RTT. Отсюда два правила:

- **клиентские задержки** k6 всегда включают ~100 мс сети, поэтому пороги в k6 заданы с этим запасом;
- **SLO оцениваются по серверным метрикам**: гистограммам `http_request_duration_seconds` самих сервисов из Prometheus в кластере (ЛР8). Они не зависят от сети и генератора.

3. Сценарии

Код — `k6/scenarios.js` и библиотека `k6/lib/{config,api}.js`. Каждый запрос помечается тегами `name` (шаблон эндпоинта, например `GET /restaurants/:id`) и `kind` (`read` / `write` / `auth`). По этим тегам заданы пороги и строится разбивка результатов.

Сценарий	Итерация	Особенности
`browse`	один запрос чтения из взвешенного набора: список (30 %), карточка (20 %), меню (15 %), столики на случайный слот (20 %), отзывы (10 %), кухни (5 %)	имитирует поток посетителей каталога; фильтры списка чередуются
`authLogin`	`POST /auth/login` тестового пользователя	CPU-ёмкая операция (bcrypt)

Сценарий	Итерация	Особенности
`booking`	бронь на случайный слот 2027 г. → немедленная отмена	у каждого VU свой зарегистрированный пользователь `k6-<run>-<vu>@load.test`; отмена освобождает слот, данные не накапливаются; `409` (слот уже занят) считается ожидаемым бизнес-ответом
`journey`	полный путь: вход → поиск → ресторан → меню → отзывы → кухни → столики → бронь → мои брони → отмена	для smoke-проверки

Ответ `409` исключён из ошибок через `http.setResponseCallback(http.expectedStatuses({min:200,max:299}, 409))`: занятый столик — нормальная ситуация для системы бронирования. Тестовые пользователи и брони после испытаний удаляются (ЛР8).

4. Профили нагрузки

Профиль	Цель	Форма нагрузки	Длительность	Макс. VU
`smoke`	проверить все эндпоинты и сценарий целиком	1 VU, 3 полных пути	< 1 мин	1
`load`	номинальная нагрузка = уровень SLO	browse: рост за 1 мин до 45/с, 3 мин держим; auth 2/с; booking 1/с (2 запроса) → **~50 запросов/с**	5 мин	130
`stress`	найти предел системы	browse ступенями 25 → 50 → 100 → 150 → 200/с по 1.5 мин; auth 2 → 6/с; booking 1/с	8 мин	150
`spike`	реакция на резкий всплеск	browse: фон 30/с → за 10 с до 180/с на 1 мин → обратно	4.5 мин	145
`soak`	стабильность во времени (утечки, рост задержек)	browse 30/с + auth 1/с + booking 1/с	10 мин	90
`hpa`	проверка автоскейлинга (ЛР8)	auth до 8 логинов/с + browse до 60/с	6.5 мин	140

Выполнены профили `smoke`, `load` и `stress` (результаты ниже), `hpa` и `spike` (в ЛР8). Профиль `soak` подготовлен, но не запускался: десятиминутная непрерывная нагрузка на общем сервере не нужна для этой работы, и её решено не создавать.

Сервер общий, поэтому для профилей действуют ограничения: прогон не длиннее 10 минут и не больше 150 VU суммарно. Интенсивность любого профиля масштабируется параметром `-e RATE=<множитель>`.

```
cd homeworks/hw10/k6
k6 run -e BASE_URL=http://<SERVER_IP>:8088/k8s -e PROFILE=load scenarios.js
k6 run -e BASE_URL=http://127.0.0.1:3000 -e PROFILE=smoke scenarios.js            # локально
# с метриками в Prometheus (remote write):
K6_PROMETHEUS_RW_SERVER_URL=http://127.0.0.1:19090/api/v1/write \
  k6 run -o experimental-prometheus-rw -e PROFILE=stress … scenarios.js
```

5. Метрики, SLI и SLO

Базовые метрики:

Метрика	Что показывает	Источник
RPS	пропускная способность	`http_reqs` (k6), `rate(http_requests_total[1m])` (сервер)
latency p50 / p95 / p99	типичный и «хвостовой» опыт пользователя	`http_req_duration` (k6), `histogram_quantile(…, http_request_duration_seconds_bucket)`
error rate	доля неуспешных ответов (5xx, таймауты; 409 не считается)	`http_req_failed` (k6), доля `status=~"5.."`
`dropped_iterations`	k6 не успел отправить запрос (не хватило VU — система не справляется)	k6
CPU / троттлинг / реплики	где узкое место	cAdvisor, kube-state-metrics

SLI (индикаторы, считаются на сервере по метрикам gateway, т. е. для всего публичного API):

```
# доступность: доля ответов без 5xx
1 - sum(rate(http_requests_total{app="gateway",status=~"5.."}[5m])) /
sum(rate(http_requests_total{app="gateway"}[5m]))
```

```
# задержка: p95 / p99 запросов через gateway
histogram_quantile(0.95, sum by (le)
(rate(http_request_duration_seconds_bucket{app="gateway"}[5m])))
```

Целевые SLO (внутренние цели команды) на номинальной нагрузке 50 запросов/с:

SLI	SLO	Порог в k6 (клиент, с RTT ~100 мс)
Доступность (доля ответов без 5xx)	≥ 99.5 % за 30 дней	`http_req_failed rate<0.01`
Задержка чтения p95	< 300 мс (сервер)	`http_req_duration{kind:read} p(95)<400`
Задержка чтения p99	< 700 мс (сервер)	`p(99)<800`
Задержка записи (бронь) p95	< 500 мс (сервер)	`http_req_duration{kind:write} p(95)<600`
Вход p95	< 600 мс (сервер; bcrypt)	`http_req_duration{name:POST /auth/login} p(95)<700`
Пропускная способность	≥ 50 запросов/с без нарушения SLO	профиль `load`

SLA (внешнее обязательство, с запасом относительно SLO): доступность публичного API **99 %** в месяц (≈ 7.3 ч простоя) и p95 < 1 с. Разница между SLO и SLA — **бюджет ошибок**. При SLO 99.5 % это ~3.6 ч в месяц, которые можно потратить на релизы и эксперименты, не нарушив обязательство перед пользователями. Если бюджет исчерпан, выкатка новых функций останавливается до восстановления.

6. Базовые метрики

Испытания проводились на облачном стенде в исходной конфигурации ЛР7: по одной реплике сервисов, gateway ×2, у всех `requests 50m` и `limits 500m`, без автоскейлинга. Сырые итоги k6 лежат в `k6/results/*.json`. Серверные метрики выгружены из Prometheus кластера за устойчивую фазу каждого прогона.

smoke (1 VU, 3 полных пути): 30 из 30 проверок, ошибок нет. Клиентские p95: чтение 138 мс, запись 177 мс, вход 270 мс.

load (номинал ~50 запросов/с, 5 минут):

```
checks_succeeded...: 100.00% 11363 out of 11363
http_req_duration..: avg=119.16ms med=108.12ms p(90)=153.66ms p(95)=197.96ms
p(99)=270.8ms max=694.61ms
```

```
  { kind:read }....: avg=115.97ms med=106.97ms p(95)=195.65ms p(99)=261.24ms
  { kind:write }...: avg=127.77ms med=122.76ms p(95)=150.95ms p(99)=313.14ms
  { login }........: avg=168.5ms  med=160.18ms p(95)=211.82ms p(99)=312.04ms
http_req_failed....: 0.00%  0 out of 11363
http_reqs..........: 11363  42.04/s (с учётом разгона; в устойчивой фазе ~49/с)
```

Серверные метрики за 3 минуты устойчивой фазы:

Сервис	**RPS**	**p50, мс**	**p95, мс**	**p99, мс**	**CPU, ядра**	**Троттлинг**
gateway (весь публичный API)	**49.0**	**6.0**	**28.1**	**91.3**	0.080 (2 пода)	0 %
catalog-service	42.6	3.0	8.7	10.4	0.096	0.1 %
review-service	26.7	2.6	5.0	31.0	0.037	0 %
reservation-service	11.2	2.8	13.1	22.9	0.021	0 %
auth-service (вход p95 = 120 мс)	6.5	3.6	94.5	165.5	0.103	2.1 %

Все сервисы вместе потребляют ~0.4 ядра, ответов 5xx нет. Удельная цена запроса: catalog ~2.3 мс CPU (список ресторанов дополнительно ходит в review за рейтингами), gateway ~1.6 мс, вход ~40–50 мс (bcrypt).

stress (ступени до 200 запросов/с на чтение, 8 минут):

```
checks_succeeded...: 100.00% 43487 out of 43487
http_req_duration..: avg=128.42ms med=108.05ms p(95)=215.25ms p(99)=421.47ms
max=15.78s
  { kind:read }....: p(95)=213.88ms p(99)=425.23ms
  { kind:write }...: p(95)=156.27ms p(99)=272.13ms
  { login }........: p(95)=237.67ms p(99)=395.97ms
http_req_failed....: 0.00%
http_reqs..........: 43487  96.64/s
dropped_iterations.: 61
```

На пике (~184 запроса/с через gateway) сервер отвечал с **p95 71 мс и p99 160 мс**, catalog — с p95 21 мс, 5xx не было. Первым признаком насыщения стал **auth-service**: при 6 входах в секунду его CPU (0.26–0.30 ядра в среднем) всплесками упирался в `limits.cpu=500m`, CFS-троттлинг составил 15 %, а p99 входа вырос до 219 мс. Одиночный выброс 15.8 с и 61 «выброшенная» итерация пришлись на короткую задержку на сетевом пути (VPN): по гистограмме gateway за весь прогон на сервере не было ни одного запроса дольше 2.5 с.

Вывод по ёмкости:

Путь	Номинал (SLO)	Проверено без нарушения SLO	Предел и узкое место
чтение каталога	45 запросов/с	~184 запроса/с (p95 на сервере 71 мс)	не достигнут: генератор ограничен 200 RPS и 150 VU; узел на пике загружен на 1.28 из 1.5 CPU — запас ~15 %
вход	2 в секунду	6 в секунду (троттлинг 15 %)	~10–12 в секунду на одну реплику auth с лимитом 500m (bcrypt) — подтверждено в ЛР8
бронирование	1 бронь/с (2 запроса)	1 бронь/с при 200 RPS чтения	не достигнут

7. Проверка SLO

SLO	Факт на номинале (load)	Запас
доступность ≥ 99.5 %	100 % (0 ошибок из 11 363)	—
чтение p95 < 300 мс (сервер)	gateway p95 28 мс	×10
чтение p99 < 700 мс (сервер)	gateway p99 91 мс	×7
запись p95 < 500 мс	клиент 151 мс, из них ~100 мс — сеть	×3
вход p95 < 600 мс	сервер 120 мс	×5
≥ 50 запросов/с без нарушения SLO	49 запросов/с в устойчивой фазе; при ~184 запросах/с SLO ещё выполняются	×3.7

Номинальная нагрузка обслуживается с большим запасом, и SLO выполняются даже при почти четырёхкратной нагрузке на чтение. Самое слабое место — вход: стоимость bcrypt делает auth-service первым кандидатом на горизонтальное масштабирование. Это и проверяется в ЛР8.

Вывод

На k6 разработаны сценарии нагрузочного тестирования ключевых эндпоинтов: просмотр каталога со взвешенным набором из шести запросов, вход, бронирование с отменой (уникальные слоты, свой пользователь у каждого VU, 409 — ожидаемый ответ) и полный пользовательский путь. Шесть профилей (smoke, load, stress, spike, soak, hpa) построены на открытой модели нагрузки и учитывают ограничения общего сервера. Определены базовые метрики (RPS, p50/p95/p99, доля ошибок) на

клиенте и на сервере. Путь от генератора откалиброван, и SLO сформулированы по серверным гистограммам, не зависящим от сети. Целевые SLO и SLA заданы вместе с бюджетом ошибок и закодированы в пороги k6. Измерения на облачном стенде показали: на номинале 50 запросов/с SLO выполняются с запасом в разы (p95 28 мс против 300 мс), путь чтения выдерживает ~184 запроса/с, а узким местом является CPU-ёмкий вход в auth-service.